ANDROID APP DEVELOPMENT USING GEN AI INTERNSHIP PROGRAM

Composable Components:
Theory, Comparison & Wireframes

A Structured Technical Report on Jetpack Compose Layout Primitives

Student Name:	Guru K.R.
Course:	Android App Development using Gen AI Internship Program
Date:	February 16, 2026
Task:	Task 2 — Composable Components

Introduction

Jetpack Compose is Android's modern, declarative UI toolkit that enables developers to build native user interfaces using a reactive programming model in Kotlin. At the heart of Jetpack Compose lies the concept of composable functions — reusable building blocks that describe how portions of the user interface should appear and behave. Unlike the traditional XML-based layout system, composables are defined entirely in code, making UI construction more intuitive, maintainable, and testable.

Among the most foundational composables are the layout primitives: **Column**, **Row**, and **Box**. These three components collectively govern how child elements are positioned and arranged on screen. A thorough understanding of their individual behaviours, use cases, and limitations is essential not only for writing effective Compose code, but also for planning and wireframing UI designs prior to implementation.

This report provides a structured theoretical overview of each layout primitive, supported by visual wireframe diagrams and a comparative analysis, forming part of the Android Internship Programme Task 2.

Section 1 — Column

Definition

A **Column** is a layout composable that arranges its child elements in a **vertical sequence**, from top to bottom. Each child occupies a distinct horizontal band, and elements are stacked one after another along the vertical axis.

Practical Use Cases

- **Login and Registration Screens:** Input fields for username, password, and action buttons are naturally stacked vertically, making Column the ideal container for form-based screens where the user progresses linearly from top to bottom.
- **Settings Screens:** A list of preference items — each composed of a label and a toggle or value — is most naturally expressed as a vertically scrolling Column, ensuring each option occupies a distinct row with full horizontal span.

Limitation

Column does not natively support horizontal arrangement of children. Placing elements side by side within a Column requires nesting a Row composable inside it, which can introduce additional complexity and nesting depth if overused across many levels of the UI hierarchy.

Example UI — Login Screen

The wireframe below illustrates a Login Screen built using Column. The screen contains a header bar followed by vertically stacked input fields for username, password, and phone number, concluding with a submit button. This arrangement reflects a natural reading order and guides the user linearly through the form. Column is highly suitable here because each element logically follows the previous one and requires full-width horizontal space.

Figure 1 — Column layout demonstrated through a Login Screen wireframe

[Insert My Wireframe – Column Layout]

Column is appropriate for this design because the login form requires a strict top-to-bottom flow. The user's eye naturally travels vertically down the screen, and each input field must occupy its own full-width row before the next element appears.

Section 2 — Row

Definition

A **Row** is a layout composable that arranges its child elements in a **horizontal sequence**, from left to right. Children are placed side by side along the horizontal axis, with optional spacing, weighting, and alignment controls.

Practical Use Cases

- **Navigation Toolbars:** App bars commonly contain a menu icon on the left, a title in the centre, and action icons (search, profile) on the right. Row provides the horizontal distribution needed to achieve this structure with precise alignment.
- **Bottom Navigation Bars:** Navigation tabs displayed at the bottom of the screen — each comprising an icon and a label — are arranged horizontally using Row, with each tab receiving an equal share of the available screen width.

Limitation

Row does not support vertical stacking of children. When the total width of child elements exceeds the available screen width, content may be clipped or overflow beyond the visible boundary. Developers must handle horizontal scrollability explicitly using the horizontalScroll modifier, adding implementation overhead.

Example UI — Application Toolbar

The wireframe below depicts a typical application toolbar laid out using Row. The toolbar contains a hamburger menu icon, a search icon, the application title, and a user profile icon — all arranged horizontally with appropriate spacing. Row is ideally suited for this design because the elements must coexist on the same horizontal plane, and the order of items is fixed from left to right.

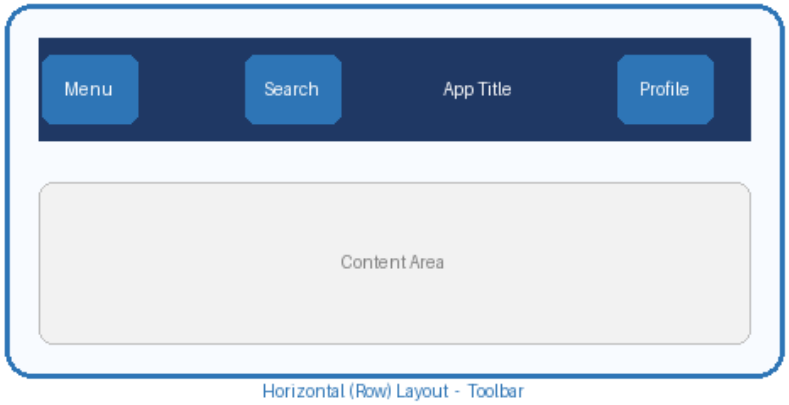

Figure 2 — Row layout demonstrated through an Application Toolbar wireframe

[Insert My Wireframe – Row Layout]

Row is appropriate for the toolbar design because all elements must appear side by side in a single horizontal strip. The directional flow of Row matches the left-to-right reading pattern expected by users for navigation controls and action items.

Section 3 — Box

Definition

A **Box** is a layout composable that stacks its child elements on top of one another along the **Z-axis (depth axis)**. Children are drawn in the order they are declared, with later children appearing above earlier ones, enabling overlapping and layered visual compositions.

Practical Use Cases

- **Badge Overlays on Profile Images:** Displaying a notification badge or an online-status indicator over a circular avatar image requires one element to be drawn on top of another — precisely the use case Box is designed for.
- **Floating Action Buttons over Content:** Positioning a Floating Action Button (FAB) in a fixed corner above a scrollable content area requires overlapping layers, which Box manages through its stacking model and alignment properties.

Limitation

Box does not provide built-in directional flow for its children. Without careful use of alignment and offset modifiers, children will all default to the top-left corner, leading to unintentional overlaps. Managing complex multi-layer layouts in Box requires precise positioning logic that can become difficult to maintain.

Example UI — Profile Image with Badge Overlay

The wireframe below illustrates a profile card where a verification badge and an online-status indicator are overlaid on top of a circular profile image using Box. The profile image forms the base layer, while the badge and status dot are positioned as overlay layers. Box is the appropriate choice because the design explicitly requires child elements to occupy the same spatial region at different depth levels.

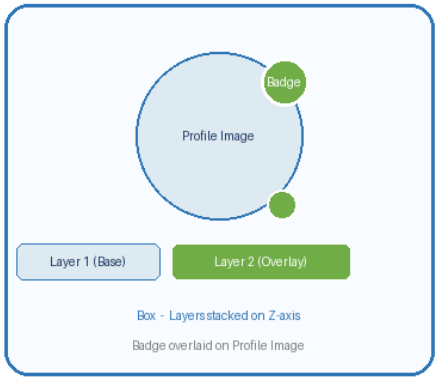

Figure 3 — Box layout demonstrated through a Profile Image with badge overlay wireframe

[Insert My Wireframe – Box Layout]

Box is appropriate here because the overlapping nature of the badge and status indicator over the profile image cannot be achieved with Column or Row. Only Box's Z-axis stacking model allows elements to occupy the same space while remaining independently positionable.

Comparison Table — Column vs Row vs Box

The following table provides a side-by-side comparison of the three layout primitives across four key criteria, serving as a quick reference guide for selecting the appropriate composable during UI design and wireframing.

Criterion	Column	Row	Box
Layout Direction	Vertical (top to bottom)	Horizontal (left to right)	Z-axis (stacked layers)
Best Use Case	Forms, settings, lists	Toolbars, tabs, icon rows	Overlays, badges, FABs

Strength	Natural vertical reading flow	Side-by-side placement	Precise layer control
Limitation	No horizontal child flow	Content clips on overflow	No directional flow; manual alignment required

Conclusion

A solid grasp of Jetpack Compose's three core layout primitives — Column, Row, and Box — is an indispensable foundation for any Android developer working in the modern Compose ecosystem. These composables are not merely technical constructs; they represent the fundamental vocabulary through which UI structure is communicated and realised.

Understanding when to use Column for vertical flows, Row for horizontal arrangements, and Box for layered compositions enables developers to plan their interfaces with precision before writing a single line of code. Wireframing with these primitives in mind allows for early detection of layout conflicts, reduces the need for costly refactoring, and aligns the development process with sound design principles.

As this internship programme progresses into more advanced topics — including state management, navigation, and custom composables — the spatial reasoning developed through this foundational exercise will prove instrumental in building scalable, maintainable, and visually coherent Android applications.